Lab 1.2. TCP Advanced Concepts

Objectives

In this lab, we will study how TCP works. Also, we will see some parameters to tune the behaviour of TCP applications. Finally, we will learn how to configure NAT with `iptables`.

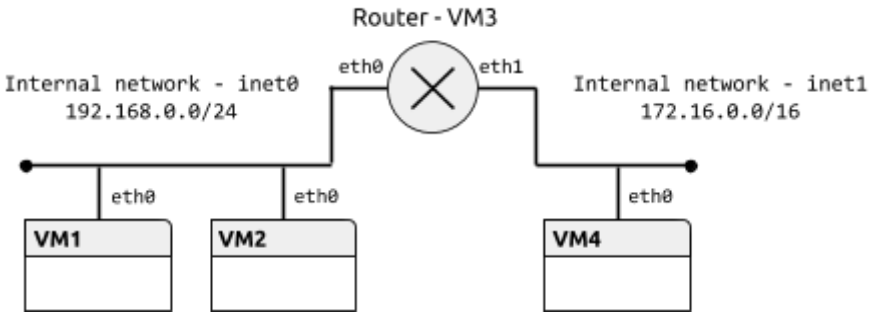
> On each exercise, provide the **commands used and their output**, the **screenshots of Wireshark captures performed**, and the **information specifically requested**.
>
> Enable **bi-directional shared clipboard** in the VMs (Devices menu).
>
> Use VirtualBox (View menu) to take **screenshots**.
>
> **Password** of user `cursoredes` is `cursoredes`.

Contents

Environment Preparation
TCP Connection Status
Introduction to the Security of TCP
TCP Options and Parameters
Network Address Translation (NAT) and Port Forwarding

Environment Preparation

We will configure the network topology shown in the following figure:



The content of the topology configuration file is the following:

```
netprefix inet
machine 1 0 0
machine 2 0 0
machine 3 0 0 1 1
machine 4 0 1
```

Finally, configure the network interfaces of all machines of the network according to the following table. After configuring all machines, check the connectivity with the `ping` command.

Machine	IPv4 Address	Comments
VM1	`192.168.0.1/24`	Add Router as the default router
VM2	`192.168.0.2/24`	Add Router as the default router
Router - VM3	`192.168.0.3/24 (eth0)` `172.16.0.3/16 (eth1)`	Enable packet forwarding
VM4	`172.16.0.4/16`	Add Router as the default router

TCP Connection Status

In this part we will use the Netcat tool (`nc` command), which allows reading from and writing to network connections. Netcat is very useful to investigate and debug the behaviour of a network at the transport layer, as it allows the user to specify many connection parameters. Moreover, we will use the `ss` command (similar to `netstat`, but more modern and complete) to see the status of network connections.

Exercise 1. Consult the man pages of `nc` and `ss`. In particular, check the following options of `ss`: `-a`, `-l`, `-t`, `-n` and `-o`. Try both commands to familiarize with their behaviour.

VM1
$man nc

```
OPTIONS SUMMARY
          Ncat 7.50 ( https://nmap.org/ncat )
          Usage: ncat [options] [hostname] [port]

          Options taking a time assume seconds. Append 'ms' for milliseconds,
          's' for seconds, 'm' for minutes, or 'h' for hours (e.g. 500ms).
            -4                         Use IPv4 only
            -6                         Use IPv6 only
            -U, --unixsock             Use Unix domain sockets only
            -C, --crlf                 Use CRLF for EOL sequence
            -c, --sh-exec <command>    Executes the given command via /bin/sh
            -e, --exec <command>       Executes the given command
                --lua-exec <filename>  Executes the given Lua script
            -g hop1[,hop2,...]         Loose source routing hop points (8 max)
            -G <n>                     Loose source routing hop pointer (4, 8,
12, ...)
            -m, --max-conns <n>        Maximum <n> simultaneous connections
            -h, --help                 Display this help screen
            -d, --delay <time>         Wait between read/writes
            -o, --output <filename>    Dump session data to a file
            -x, --hex-dump <filename>  Dump session data as hex to a file
            -i, --idle-timeout <time>  Idle read/write timeout
 Manual page nc(1) line 27 (press h for help or q to quit)
```

VM1
$man ss

```
SYNOPSIS
       ss [options] [ FILTER ]

DESCRIPTION
       ss  is  used  to  dump socket statistics. It allows showing information
       similar to netstat.  It can display more  TCP  and  state  informations
       than other tools.

OPTIONS
       When no option is used ss displays a list of open non-listening sockets
       (e.g. TCP/UNIX/UDP) that have established connection.

       -h, --help
              Show summary of options.

       -V, --version
              Output version information.

       -H, --no-header
              Suppress header line.

       -n, --numeric
              Do not try to resolve service names.

       -r, --resolve
              Try to resolve numeric address/ports.

       -a, --all
              Display both listening and non-listening  (for  TCP  this  means
              established connections) sockets.

       -l, --listening
              Display only listening sockets (these are omitted by default).

       -o, --options
              Show timer information.

       -e, --extended
              Show detailed socket information

       -m, --memory
```

Exercise 2. (LISTEN) Start a TCP server in VM1 on port 7777 using command `nc -l 7777`. Check the connection status of the server with command `ss -tln`. Start another server in VM1 using command `nc -l 192.168.0.1 7776`. Observe the difference between both servers using `ss`. Check that it is not possible to connect to the second server from VM1 with `localhost` as destination address using command `nc localhost 7776`.

VM1
$ nc -l 7777
$ nc -l 192.168.0.1 7776
$ss -tln
*7777: LISTEN 0 10 :::7777 :::**
*7776: LISTEN 0 10 192.168.0.1:7776 *:**

$ sudo nc -l 192.168.0.1 7776 (error)
Localhost again: Ncat: bind to 192.168.0.1:7776: Address already in use. QUITTING.

Exercise 3. (ESTABLISHED) Start a client connection in VM2 to the first server started in the previous exercise using command `nc 192.168.0.1 7777`.

- Check the connection status and identify its parameters (IP address and port) using command `ss -tn`.
- Start a capture with Wireshark. Interchange a single character with the client and observe the messages interchanged (specially, sequence numbers, acknowledgement numbers and TCP flags) and calculate how many bytes (and messages) were needed.

```
VM2
$ nc 192.168.0.1 7777
h

VM1
$ ss -tn
State      Rec  Send  Local address:Port          Peer Address:port
ESTAB      0    0     192.168.0.1:7777            192.168.0.2:50014
```

No.	Time	Source	Destination	Protoc	Lengt	Info
1	0.00000000	192.168.0.2	192.168.0.1	TCP	68	50040 > cbt [PSH, ACK] Seq=1 Ack=1 Win=229 Len=2 TSval=14322458 TSecr=14269002
2	0.00002237	192.168.0.1	192.168.0.2	TCP	66	cbt > 50040 [ACK] Seq=1 Ack=3 Win=227 Len=0 TSval=14282954 TSecr=14322458

```
▸ Frame 1: 68 bytes on wire (544 bits), 68 bytes captured (544 bits) on interface 0
▸ Ethernet II, Src: CadmusCo_be:d7:65 (08:00:27:be:d7:65), Dst: CadmusCo_23:11:ec (08:00:27:23:11:ec)
▸ Internet Protocol Version 4, Src: 192.168.0.2 (192.168.0.2), Dst: 192.168.0.1 (192.168.0.1)
▾ Transmission Control Protocol, Src Port: 50040 (50040), Dst Port: cbt (7777), Seq: 1, Ack: 1, Len: 2
    Source port: 50040 (50040)
    Destination port: cbt (7777)
    [Stream index: 0]
    Sequence number: 1    (relative sequence number)
    [Next sequence number: 3    (relative sequence number)]
    Acknowledgment number: 1    (relative ack number)
    Header length: 32 bytes
  ▾ Flags: 0x018 (PSH, ACK)
      000. .... .... = Reserved: Not set
      ...0 .... .... = Nonce: Not set
      .... 0... .... = Congestion Window Reduced (CWR): Not set
      .... .0.. .... = ECN-Echo: Not set
      .... ..0. .... = Urgent: Not set
      .... ...1 .... = Acknowledgment: Set
      .... .... 1... = Push: Set
      .... .... .0.. = Reset: Not set
      .... .... ..0. = Syn: Not set
      .... .... ...0 = Fin: Not set
    Window size value: 229
    [Calculated window size: 229]
    [Window size scaling factor: -1 (unknown)]
  ▸ Checksum: 0x3b0b [validation disabled]
  ▸ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  ▸ [SEQ/ACK analysis]
```

2 messages and 133 bytes were needed

Exercise 4. (TIME-WAIT) Close the connection in the client (with `Ctrl+C`) and check the connection state using `ss -tan`. Use option `-o` to determine the value of the TIME-WAIT timer.

```
VM1
$Ctrl + C
State       Rec  Send  Local address:Port          Peer Address:port
TIME-WAIT   0    0     192.168.0.2:50042           192.168.0.1:7777
timer:(timewait,6.073ms,0)
```

Exercise 5. (SYN-SENT and SYN-RCVD) It is possible to filter packets depending on TCP flags of segments with option `--tcp-flags` of `iptables` (see `man iptables-extensions`). Using this option:

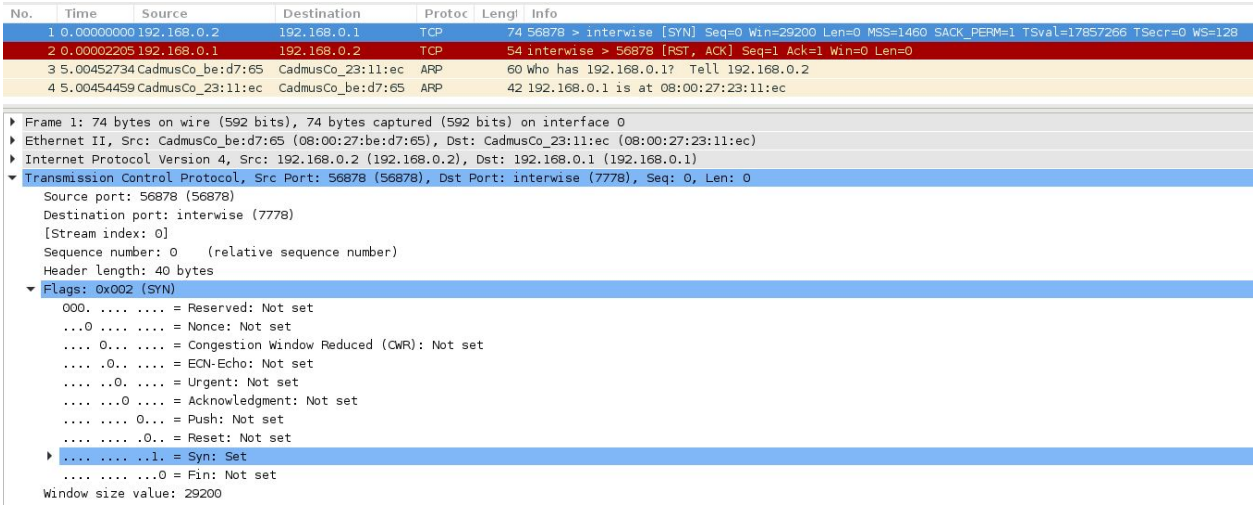
- Set a rule in the server (VM1) to block one message of the TCP handshake so that the client (VM2) remains in SYN-SENT state. Check the result using `ss -tan` in the client.
- Remove the previous rule and set another one in the client to block one message of the TCP handshake so that the server remains in SYN-RCVD state. Check the result using `ss -tan` in the server. This rule would also make the server remain in LAST-ACK state after closing the connection in the client. Use option `-o` of `ss` to determine how many retransmissions are done and how often.

```
VM1
$nc -l 7777
$ sudo iptables -A INPUT --protocol tcp --tcp-flags ALL SYN -j DROP

VM2
$nc 192.168.0.1 7777
$a
$ss -tan

VM1
$ sudo iptables -F

State       Rec  Send Local address:Port        Peer Address:port
SYN-SENT    0    1    192.168.0.2:60146         192.168.0.1:7777
```

```
VM2
$sudo iptables -A OUTPUT -p tcp --tcp-flags ALL ACK -j DROP

VM1
$ss -tan

VM2
$sudo iptables -F

State       Rec  Send Local address:Port        Peer Address:port
SYN-RECV    0    0    192.168.0.1:7777          192.168.0.2:60184
```

Exercise 6. Start a capture with Wireshark. Try a connection to a closed port in the server (e.g. 7778) and observe the TCP segments interchanged, specially the TCP flags.

```
VM1
$nc 192.168.0.1 7778
Ncat: Connection refused.
```



Introduction to the Security of TCP

Different aspects of TCP can be used to compromise system security. In this part, we are going to study two of them: TCP SYN flood DoS attacks and port scanning techniques.

Exercise 7. A TCP SYN flood attack consists in saturating a server with the massive sending of SYN messages.

- (Client VM2) To prevent that the attacker replied to SYN+ACK messages from the server with a RST message (that would free the connection), block these messages with `iptables`.
- (Client VM2) Use the `hping3` command (consult the `man` page) to send SYN messages to port 22 of the server (`ssh`) as fast as possible (*flood*).
- (Server VM1) Analyze the behaviour of the machine, in terms of received packets. Check if it is possible to connect to the `ssh` service.

Repeat the attack, disabling the SYN cookies mechanism in the server with the `sysctl` command (`net.ipv4.tcp_syncookies` parameter).

VM2
$ iptables -A OUTPUT -p tcp --tcp-flags ALL RST -j DROP
(Another option: -A INPUT -p tcp -m state --state NEW -j DROP)
$ hping3 -p 22 -S --flood 192.168.0.1

VM1
$ nc localhost 22

Copy the iptables and hping3 commands. Describe the behaviour of the machine with and without the SYN cookies mechanism.

Exercise 8. (CONNECT technique) Netcat can scan ports using the CONNECT technique, which tries to establish a connection with a given port. Depending on the answer (SYN+ACK or RST), it is possible to determine if there is a process listening.

- (Server VM1) Start a server in port 7777.
- (Client VM2) Scan, one by one, the port rank 7775-7780 using `nc`. Use options for exploration (`-z`) and verbose output (`-v`). **Note:** The installed version of `nc` does not support port ranges.

```
VM1
$nc -l 7777

VM2
(WIth ranges would be: nc -z -v 192.168.0.1 7775-7780)
$nc -z -v 192.168.0.1 7775
$nc -z -v 192.168.0.1 7776
$nc -z -v 192.168.0.1 7777
$nc -z -v 192.168.0.1 7778
$nc -z -v 192.168.0.1 7779
$nc -z -v 192.168.0.1 7780


Copy the nc commands used and their output.
Ncat: Connection refused.
Ncat: Connection refused.

Ncat: Connected to 192.168.0.1:7777.
Ncat: 0 bytes sent, 0 bytes received in 0.01 seconds.

Ncat: Connection refused.
Ncat: Connection refused.
Ncat: Connection refused.
```

Optional. The Nmap tool can perform different port scan techniques, using more efficient strategies. These techniques (SYN stealth, ACK stealth, FIN-ACK stealth…) are based on the operation of TCP. Consult the `man` page of `nmap` (section PORT SCANNING TECHNIQUES) and use different techniques to scan server ports. Observe the messages interchanged with Wireshark.

TCP Options and Parameters

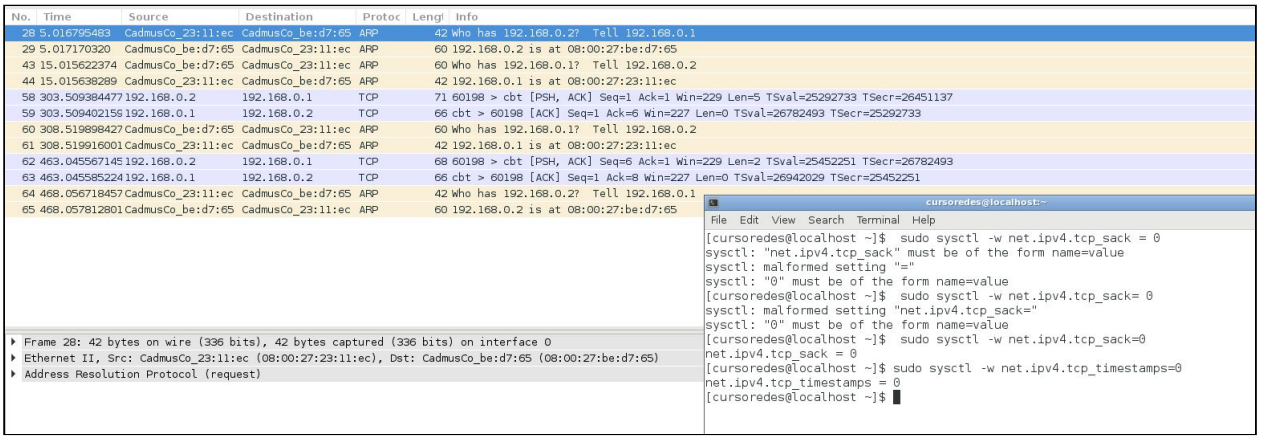
The behaviour of a TCP connection can be controlled with several options that are included in the header of SYN messages and can be configured by the operating system by means of kernel parameters.

Exercise 9. Using the `sysctl` command and the recommended bibliography, complete the following table with parameters that allow changing some options of TCP:

Kernel parameter	Purpose	Default value
net.ipv4.tcp_window_scaling	The TCP window scale option is an option to increase the receive window size allowed in Transmission Control Protocol	1
net.ipv4.tcp_timestamps	Turn timestamps off to reduce performance spikes related to timestamp generation, turns on in other case.	1
net.ipv4.tcp_sack	Enable Selective-ACKs to confirme out of order packages.	1

Exercise 10. Start a capture with Wireshark Start a server on port 7777 and make a connection from the client VM. Study the value of the options interchanged during the connection. Vary some of the previous parameters (e.g. don't use selective ACKs) and observe the result in a new connection.

```
VM1 (Server)
$nc -l 7777

VM2
$nc 192.168.0.1 7777
$h

VM1
$ sudo sysctl -w net.ipv4.tcp_sack=0
$sudo sysctl -w net.ipv4.tcp_timestamps=0

VM2
$a
```



Exercise 11. Using the `sysctl` command and the recommended bibliography, complete the following table with kernel parameters that can be used to configure the keepalive timer:

Kernel parameter	**Purpose**	**Default value**
`net.ipv4.tcp_keepalive_time`	Indicates the lifetime of a TCP connection in seconds.	7200
`net.ipv4.tcp_keepalive_probes`	Determines the frequency of keepalive packages before deciding a connection has been interrupted.	9
`net.ipv4.tcp_keepalive_intvl`	Determines the length of an answer for each keepalive package.	75

Network Address Translation (NAT) and Port Forwarding

In this part, we will assume that the network connection in Router with VM4 is public and can't route traffic from `192.168.0.0/24`. Also, we will suppose that the IP address of Router is dynamic.

Exercise 12. Configure dynamic address translation in Router :

- (Router) Using `iptables`, configure Router to perform SNAT (masquerade) on interface `eth1`. Start a packet capture with Wireshark.
- (VM1) Check the connectivity between VM1 and VM4 with the `ping` command.
- (Router) Analyze with Wireshark the traffic, specially source and destination ports and IP addresses in both networks.

```
Router
$ sudo iptables -t nat -A POSTROUTING -o eth1 -j MASQUERADE

VM1
$ping 172.16.0.4
```

No.	Time	Source	Destination	Protoc	Leng	Info
1	0.00000000	192.168.0.1	172.16.0.4	ICMP	98	Echo (ping) request id=0x1c97, seq=1/256, ttl=64 (reply in 4)
2	0.00005222	172.16.0.3	172.16.0.4	ICMP	98	Echo (ping) request id=0x1c97, seq=1/256, ttl=63 (reply in 3)
3	0.00047987	172.16.0.4	172.16.0.3	ICMP	98	Echo (ping) reply id=0x1c97, seq=1/256, ttl=64 (request in 2)
4	0.00049139	172.16.0.4	192.168.0.1	ICMP	98	Echo (ping) reply id=0x1c97, seq=1/256, ttl=63 (request in 1)
5	1.00096285	192.168.0.1	172.16.0.4	ICMP	98	Echo (ping) request id=0x1c97, seq=2/512, ttl=64 (reply in 8)
6	1.00098598	172.16.0.3	172.16.0.4	ICMP	98	Echo (ping) request id=0x1c97, seq=2/512, ttl=63 (reply in 7)
7	1.00143842	172.16.0.4	172.16.0.3	ICMP	98	Echo (ping) reply id=0x1c97, seq=2/512, ttl=64 (request in 6)
8	1.00145453	172.16.0.4	192.168.0.1	ICMP	98	Echo (ping) reply id=0x1c97, seq=2/512, ttl=63 (request in 5)
9	2.00159167	192.168.0.1	172.16.0.4	ICMP	98	Echo (ping) request id=0x1c97, seq=3/768, ttl=64 (reply in 12)
10	2.00161741	172.16.0.3	172.16.0.4	ICMP	98	Echo (ping) request id=0x1c97, seq=3/768, ttl=63 (reply in 11)
11	2.00205411	172.16.0.4	172.16.0.3	ICMP	98	Echo (ping) reply id=0x1c97, seq=3/768, ttl=64 (request in 10)
12	2.00207072	172.16.0.4	192.168.0.1	ICMP	98	Echo (ping) reply id=0x1c97, seq=3/768, ttl=63 (request in 9)
13	5.00884312	CadmusCo_44:08:52	CadmusCo_19:68:90	ARP	42	Who has 172.16.0.4? Tell 172.16.0.3
14	5.00885899	CadmusCo_ca:55:29	CadmusCo_23:11:ec	ARP	42	Who has 192.168.0.1? Tell 192.168.0.3
15	5.00887887	CadmusCo_23:11:ec	CadmusCo_ca:55:29	ARP	60	Who has 192.168.0.3? Tell 192.168.0.1
16	5.00888796	CadmusCo_ca:55:29	CadmusCo_23:11:ec	ARP	42	192.168.0.3 is at 08:00:27:ca:55:29
17	5.00934864	CadmusCo_23:11:ec	CadmusCo_ca:55:29	ARP	60	192.168.0.1 is at 08:00:27:23:11:ec
18	5.00947445	CadmusCo_19:68:90	CadmusCo_44:08:52	ARP	60	172.16.0.4 is at 08:00:27:19:68:90

Exercise 13. Which parameter is used, instead of source port, to match Echo replies with Echo requests? Check the output of command `conntrack -L` or, alternatively, file `/proc/net/nf_conntrack`.

```
VM3:
$ sudo conntrack -L
```

Exercise 14. Configure access to a server in the private network:

- (Router) Using `iptables`, forward connections (DNAT) from Router's port 80 to VM1's port 7777. Start a packet capture with Wireshark.
- (VM1) Start the server on port 7777 with nc.
- (VM4) Connect to port 80 of Router with nc and check the result in VM1.
- (Router) Analyze with Wireshark the traffic, specially source and destination ports and IP addresses in both networks.

```
Router
$ sudo iptables -t nat -A PREROUTING -d 172.16.0.3 -p tcp --dport 80 -j DNAT --to 192.168.0.1:7777
```

No.	Time	Source	Destination	Protoc	Leng	Info
1	0.00000000	172.16.0.4	192.168.0.1	TCP	68	53492 → cbt [PSH, ACK] Seq=1 Ack=1 Win=29200 Len=2 TSval=113695255 TSecr=113682612
2	0.00041210	192.168.0.1	172.16.0.4	TCP	66	cbt → 53492 [ACK] Seq=1 Ack=3 Win=28960 Len=0 TSval=113744034 TSecr=113695255
3	5.01080464	CadmusCo_5c:90:c2	CadmusCo_4c:af:03	ARP	42	Who has 192.168.0.1? Tell 192.168.0.3
4	5.01080392	CadmusCo_4c:af:03	CadmusCo_5c:90:c2	ARP	60	Who has 192.168.0.3? Tell 192.168.0.1
5	5.01082126	CadmusCo_5c:90:c2	CadmusCo_4c:af:03	ARP	42	192.168.0.3 is at 08:00:27:5c:90:c2
6	5.01072411	CadmusCo_4c:af:03	CadmusCo_5c:90:c2	ARP	60	192.168.0.1 is at 08:00:27:4c:af:03